

# Mini Project W28 - Thứ 6 chiều: Bank Account System

## Mục tiêu Mini Project

- Xây dựng hệ thống quản lý tài khoản ngân hàng hoàn chỉnh
- Áp dụng tất cả kiến thức đã học: OOP, Functions, Lists, Dictionaries
- Tạo giao diện người dùng đơn giản
- Thực hành thiết kế và tổ chức code

## Kiến thức sẽ sử dụng

- Classes và Objects (buổi sáng)
- Methods và **init** (buổi sáng)
- Functions (tuần trước)
- Lists và Dictionaries (tuần 27)
- Loops và If/else (tuần 26)
- Input/Output (tuần 26)

## Phase 1: Hoàn thiện Bank Account Class

Đầu tiên, chúng ta sẽ xây dựng class BankAccount hoàn chỉnh với đầy đủ tính năng:

```
In [ ]: import datetime

class BankAccount:
    # Class variables
    bank_name = "Python Bank"
    next_account_number = 1001

    def __init__(self, account_holder, initial_balance=0):
        # Instance variables
        self.account_number = BankAccount.next_account_number
        BankAccount.next_account_number += 1

        self.account_holder = account_holder
        self.balance = initial_balance
        self.transaction_history = []

        # Ghi lại giao dịch tạo tài khoản
        self.add_transaction("Tạo tài khoản", initial_balance)
        print(f"✅ Tạo tài khoản thành công!")
        print(f"Số tài khoản: {self.account_number}")
        print(f"Chủ tài khoản: {self.account_holder}")
```

```

print(f"Số dư ban đầu: {self.balance:,} VND")

def add_transaction(self, transaction_type, amount):
    """Thêm giao dịch vào lịch sử"""
    transaction = {
        "time": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "type": transaction_type,
        "amount": amount,
        "balance_after": self.balance
    }
    self.transaction_history.append(transaction)

def deposit(self, amount):
    """Nạp tiền vào tài khoản"""
    if amount <= 0:
        print("❌ Số tiền nạp phải lớn hơn 0!")
        return False

    self.balance += amount
    self.add_transaction("Nạp tiền", amount)
    print(f"✅ Nạp {amount:,} VND thành công!")
    print(f"Số dư mới: {self.balance:,} VND")
    return True

def withdraw(self, amount):
    """Rút tiền từ tài khoản"""
    if amount <= 0:
        print("❌ Số tiền rút phải lớn hơn 0!")
        return False

    if amount > self.balance:
        print(f"❌ Không đủ số dư! Số dư hiện tại: {self.balance:,} VND")
        return False

    self.balance -= amount
    self.add_transaction("Rút tiền", -amount)
    print(f"✅ Rút {amount:,} VND thành công!")
    print(f"Số dư mới: {self.balance:,} VND")
    return True

def transfer(self, other_account, amount):
    """Chuyển tiền sang tài khoản khác"""
    if amount <= 0:
        print("❌ Số tiền chuyển phải lớn hơn 0!")
        return False

    if amount > self.balance:
        print(f"❌ Không đủ số dư! Số dư hiện tại: {self.balance:,} VND")
        return False

    # Thực hiện chuyển tiền
    self.balance -= amount
    other_account.balance += amount

    # Ghi lại giao dịch cho cả 2 tài khoản
    self.add_transaction(f"Chuyển tiền đến TK {other_account.account_number}",

```

```

other_account.add_transaction(f"Nhận tiền từ TK {self.account_number}", amo

print(f"✅ Chuyển {amount:,} VND thành công!")
print(f"Từ: {self.account_holder} (TK {self.account_number})")
print(f"Đến: {other_account.account_holder} (TK {other_account.account_num
print(f"Số dư mới của bạn: {self.balance:,} VND")
return True

def get_balance(self):
    """Lấy số dư hiện tại"""
    return self.balance

def get_account_info(self):
    """Lấy thông tin tài khoản"""
    print(f"\n=== THÔNG TIN TÀI KHOẢN ===")
    print(f"Ngân hàng: {BankAccount.bank_name}")
    print(f"Số tài khoản: {self.account_number}")
    print(f"Chủ tài khoản: {self.account_holder}")
    print(f"Số dư hiện tại: {self.balance:,} VND")
    print(f"Tổng số giao dịch: {len(self.transaction_history)}")

def show_transaction_history(self, limit=10):
    """Hiển thị lịch sử giao dịch"""
    print(f"\n=== LỊCH SỬ GIAO DỊCH (Top {limit}) ===")
    if not self.transaction_history:
        print("Chưa có giao dịch nào.")
        return

    # Hiển thị các giao dịch gần nhất
    recent_transactions = self.transaction_history[-limit:]
    for i, trans in enumerate(reversed(recent_transactions), 1):
        amount_str = f"{trans['amount']:,}" if trans['amount'] >= 0 else f"tra
        print(f"{i:2d}. {trans['time']} | {trans['type']:<25} | {amount_str:>15

# Test class BankAccount
print("=== TEST BANK ACCOUNT CLASS ===")
account1 = BankAccount("Nguyễn Văn A", 1000000)
account2 = BankAccount("Trần Thị B", 500000)

```

## TODO 1: Test các tính năng cơ bản

Hãy test các method của BankAccount class:

```

In [ ]: # TODO 1: Test các tính năng cơ bản
print("\n=== TEST CÁC TÍNH NĂNG ===")

# Test thông tin tài khoản
account1.get_account_info()

# Test nạp tiền
print("\n--- Test nạp tiền ---")
# Viết code test nạp tiền ở đây

# Test rút tiền
print("\n--- Test rút tiền ---")

```

```
# Viết code test rút tiền ở đây

# Test chuyển tiền
print("\n--- Test chuyển tiền ---")
# Viết code test chuyển tiền ở đây

# Test lịch sử giao dịch
print("\n--- Test lịch sử giao dịch ---")
# Viết code hiển thị lịch sử ở đây
```

## Phase 2: Bank Manager Class

Tạo class để quản lý nhiều tài khoản:

```
In [ ]: class BankManager:
    def __init__(self):
        self.accounts = {} # Dictionary để lưu tài khoản theo số tài khoản
        self.current_user = None # Tài khoản đang đăng nhập

    def create_account(self, name, initial_balance=0):
        """Tạo tài khoản mới"""
        account = BankAccount(name, initial_balance)
        self.accounts[account.account_number] = account
        return account

    def find_account(self, account_number):
        """Tìm tài khoản theo số"""
        return self.accounts.get(account_number)

    def login(self, account_number):
        """Đăng nhập vào tài khoản"""
        account = self.find_account(account_number)
        if account:
            self.current_user = account
            print(f"✅ Đăng nhập thành công!")
            print(f"Chào mừng {account.account_holder}!")
            return True
        else:
            print(f"❌ Không tìm thấy tài khoản số {account_number}!")
            return False

    def logout(self):
        """Đăng xuất"""
        if self.current_user:
            print(f"Tạm biệt {self.current_user.account_holder}!")
            self.current_user = None
        else:
            print("Chưa có ai đăng nhập!")

    def show_all_accounts(self):
        """Hiển thị tất cả tài khoản (chỉ thông tin cơ bản)"""
        print("\n=== DANH SÁCH TÀI KHOẢN ===")
        if not self.accounts:
            print("Chưa có tài khoản nào.")
```

```

        return

    print(f"{'STK':<6} | {'Chủ tài khoản':<20} | {'Số dư (VND)':<15}")
    print("-" * 50)
    for account in self.accounts.values():
        print(f"{account.account_number:<6} | {account.account_holder:<20} | {a

def get_total_money(self):
    """Tính tổng tiền trong ngân hàng"""
    total = sum(account.balance for account in self.accounts.values())
    return total

# Tạo bank manager
bank = BankManager()
print("=== KHỞI TẠO BANK MANAGER ===")
print("Bank Manager đã được tạo!")

```

## TODO 2: Test Bank Manager

Tạo một vài tài khoản và test các tính năng của Bank Manager:

```

In [ ]: # TODO 2: Test Bank Manager
print("\n=== TEST BANK MANAGER ===")

# Tạo một vài tài khoản
print("\n--- Tạo tài khoản ---")
# Viết code tạo 3 tài khoản khác nhau ở đây

# Hiển thị danh sách tài khoản
print("\n--- Danh sách tài khoản ---")
# Viết code hiển thị tất cả tài khoản ở đây

# Test đăng nhập
print("\n--- Test đăng nhập ---")
# Viết code test đăng nhập ở đây

# Test tổng tiền trong ngân hàng
print("\n--- Tổng tiền trong ngân hàng ---")
# Viết code hiển thị tổng tiền ở đây

```

## Phase 3: Utility Functions

Tạo các function hỗ trợ cho giao diện người dùng:

```

In [ ]: def get_number_input(prompt, min_value=0):
    """Lấy input số từ người dùng với validation"""
    while True:
        try:
            value = float(input(prompt))
            if value < min_value:
                print(f"❌ Giá trị phải >= {min_value}")
                continue

```

```

        return int(value) if value == int(value) else value
    except ValueError:
        print("❌ Vui lòng nhập số hợp lệ!")

def get_string_input(prompt):
    """Lấy input string từ người dùng"""
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("❌ Vui lòng nhập thông tin!")

def print_separator(title=""):
    """In dòng phân cách đẹp"""
    if title:
        print(f"\n{'='*20} {title} {'='*20}")
    else:
        print("\n" + "="*60)

def wait_for_enter():
    """Chờ người dùng nhấn Enter"""
    input("\nNhấn Enter để tiếp tục...")

def clear_screen():
    """Xóa màn hình (giả lập)"""
    print("\n" * 3)
    print("=" * 60)
    print(f"🏠 {BankAccount.bank_name} - Hệ thống quản lý tài khoản")
    print("=" * 60)

# Test utility functions
print("=== TEST UTILITY FUNCTIONS ===")
print_separator("TEST")
clear_screen()

```

## Phase 4: Main Menu System

Tạo hệ thống menu chính:

```

In [ ]: def show_main_menu():
        """Hiển thị menu chính"""
        print("\n📋 MENU CHÍNH:")
        print("1. Tạo tài khoản mới")
        print("2. Đăng nhập")
        print("3. Xem danh sách tài khoản")
        print("4. Thống kê ngân hàng")
        print("0. Thoát chương trình")

def show_user_menu(account):
        """Hiển thị menu người dùng"""
        print(f"\n👤 MENU CỦA {account.account_holder} (TK: {account.account_number}):")
        print("1. Xem thông tin tài khoản")
        print("2. Nạp tiền")
        print("3. Rút tiền")

```

```

print("4. Chuyển tiền")
print("5. Xem lịch sử giao dịch")
print("6. Đăng xuất")

def create_new_account(bank):
    """Tạo tài khoản mới"""
    print_separator("TẠO TÀI KHOẢN MỚI")

    name = get_string_input("Nhập tên chủ tài khoản: ")
    initial_balance = get_number_input("Nhập số dư ban đầu (VND): ", 0)

    account = bank.create_account(name, initial_balance)
    print(f"\n🎉 Tài khoản được tạo thành công với số: {account.account_number}")
    wait_for_enter()

def login_account(bank):
    """Đăng nhập tài khoản"""
    print_separator("ĐĂNG NHẬP")

    if not bank.accounts:
        print("❌ Chưa có tài khoản nào trong hệ thống!")
        wait_for_enter()
        return

    account_number = get_number_input("Nhập số tài khoản: ", 1)

    if bank.login(account_number):
        wait_for_enter()
        user_menu_loop(bank)
    else:
        wait_for_enter()

def view_all_accounts(bank):
    """Xem danh sách tài khoản"""
    print_separator("DANH SÁCH TÀI KHOẢN")
    bank.show_all_accounts()
    wait_for_enter()

def show_bank_statistics(bank):
    """Hiển thị thống kê ngân hàng"""
    print_separator("THỐNG KÊ NGÂN HÀNG")
    print(f"🏦 Ngân hàng: {BankAccount.bank_name}")
    print(f"📊 Tổng số tài khoản: {len(bank.accounts)}")
    print(f"💰 Tổng tiền trong ngân hàng: {bank.get_total_money():,} VND")

    if bank.accounts:
        max_balance = max(acc.balance for acc in bank.accounts.values())
        min_balance = min(acc.balance for acc in bank.accounts.values())
        avg_balance = bank.get_total_money() / len(bank.accounts)

        print(f"📈 Số dư cao nhất: {max_balance:,} VND")
        print(f"📉 Số dư thấp nhất: {min_balance:,} VND")
        print(f"📊 Số dư trung bình: {avg_balance:,.0f} VND")

    wait_for_enter()

```

```
# Test menu functions
print("=== TEST MENU FUNCTIONS ===")
show_main_menu()
print("\nMenu functions đã được tạo!")
```

## TODO 3: Hoàn thiện User Menu Functions

Tạo các function cho menu người dùng:

```
In [ ]: # TODO 3: Hoàn thiện User Menu Functions

def view_account_info(account):
    """Xem thông tin tài khoản"""
    print_separator("THÔNG TIN TÀI KHOẢN")
    # Viết code hiển thị thông tin tài khoản ở đây
    pass

def deposit_money(account):
    """Nạp tiền"""
    print_separator("NẠP TIỀN")
    # Viết code nạp tiền ở đây
    pass

def withdraw_money(account):
    """Rút tiền"""
    print_separator("RÚT TIỀN")
    # Viết code rút tiền ở đây
    pass

def transfer_money(account, bank):
    """Chuyển tiền"""
    print_separator("CHUYỂN TIỀN")
    # Viết code chuyển tiền ở đây
    # Gợi ý: cần tìm tài khoản đích trong bank.accounts
    pass

def view_transaction_history(account):
    """Xem lịch sử giao dịch"""
    print_separator("LỊCH SỬ GIAO DỊCH")
    # Viết code hiển thị lịch sử ở đây
    pass

print("User menu functions template đã được tạo!")
print("Hãy hoàn thiện các function trong TODO 3")
```

## Phase 5: Main Program Loop

Tạo vòng lặp chính của chương trình:

```
In [ ]: def user_menu_loop(bank):
    """Vòng lặp menu người dùng"""
    account = bank.current_user
```



```

while True:
    clear_screen()
    show_user_menu(account)

    choice = input("\nChọn chức năng (0-6): ").strip()

    if choice == "1":
        view_account_info(account)
    elif choice == "2":
        deposit_money(account)
    elif choice == "3":
        withdraw_money(account)
    elif choice == "4":
        transfer_money(account, bank)
    elif choice == "5":
        view_transaction_history(account)
    elif choice == "6":
        print("Đăng xuất thành công!")
        bank.logout()
        break
    else:
        print("❌ Lựa chọn không hợp lệ!")
        wait_for_enter()

def main_program():
    """Chương trình chính"""
    bank = BankManager()

    # Tạo sẵn một vài tài khoản demo
    bank.create_account("Demo User 1", 1000000)
    bank.create_account("Demo User 2", 500000)

    print("\n🎉 Chào mừng đến với Python Bank!")
    print("Đã tạo sẵn 2 tài khoản demo để test:")
    bank.show_all_accounts()

    while True:
        clear_screen()
        show_main_menu()

        choice = input("\nChọn chức năng (0-4): ").strip()

        if choice == "1":
            create_new_account(bank)
        elif choice == "2":
            login_account(bank)
        elif choice == "3":
            view_all_accounts(bank)
        elif choice == "4":
            show_bank_statistics(bank)
        elif choice == "0":
            print("\n👋 Cảm ơn bạn đã sử dụng Python Bank!")
            print("Hẹn gặp lại!")
            break
        else:

```

```

        print("❌ Lựa chọn không hợp lệ!")
        wait_for_enter()

# Chạy chương trình
if __name__ == "__main__":
    main_program()

```

## Phase 6: Hoàn thiện TODO Functions

Bây giờ hãy hoàn thiện tất cả các TODO functions:

```

In [ ]: # Hoàn thiện TODO 1: Test các tính năng cơ bản
def test_basic_features():
    print("\n=== TEST CÁC TÍNH NĂNG CỦA BANKACCOUNT ===")

    # Tạo tài khoản test
    account1 = BankAccount("Nguyễn Văn A", 1000000)
    account2 = BankAccount("Trần Thị B", 500000)

    # Test thông tin tài khoản
    account1.get_account_info()

    # Test nạp tiền
    print("\n--- Test nạp tiền ---")
    account1.deposit(200000)
    account1.deposit(-50000) # Test số âm

    # Test rút tiền
    print("\n--- Test rút tiền ---")
    account1.withdraw(300000)
    account1.withdraw(2000000) # Test rút quá số dư

    # Test chuyển tiền
    print("\n--- Test chuyển tiền ---")
    account1.transfer(account2, 100000)

    # Test lịch sử giao dịch
    print("\n--- Test lịch sử giao dịch ---")
    account1.show_transaction_history()
    account2.show_transaction_history()

# Hoàn thiện TODO 2: Test Bank Manager
def test_bank_manager():
    print("\n=== TEST BANK MANAGER ===")

    bank = BankManager()

    # Tạo một vài tài khoản
    print("\n--- Tạo tài khoản ---")
    acc1 = bank.create_account("Alice Wonderland", 2000000)
    acc2 = bank.create_account("Bob Builder", 1500000)
    acc3 = bank.create_account("Charlie Chocolate", 800000)

    # Hiển thị danh sách tài khoản

```

```

print("\n--- Danh sách tài khoản ---")
bank.show_all_accounts()

# Test đăng nhập
print("\n--- Test đăng nhập ---")
bank.login(acc1.account_number)
bank.login(9999) # Test đăng nhập sai số TK

# Test tổng tiền trong ngân hàng
print("\n--- Tổng tiền trong ngân hàng ---")
total = bank.get_total_money()
print(f"💰 Tổng tiền: {total:,} VND")

# Hoàn thiện TODO 3: User Menu Functions
def view_account_info(account):
    """Xem thông tin tài khoản"""
    print_separator("THÔNG TIN TÀI KHOẢN")
    account.get_account_info()
    wait_for_enter()

def deposit_money(account):
    """Nạp tiền"""
    print_separator("NẠP TIỀN")
    print(f"Số dư hiện tại: {account.balance:,} VND")

    amount = get_number_input("Nhập số tiền muốn nạp (VND): ", 1)
    account.deposit(amount)
    wait_for_enter()

def withdraw_money(account):
    """Rút tiền"""
    print_separator("RÚT TIỀN")
    print(f"Số dư hiện tại: {account.balance:,} VND")

    amount = get_number_input("Nhập số tiền muốn rút (VND): ", 1)
    account.withdraw(amount)
    wait_for_enter()

def transfer_money(account, bank):
    """Chuyển tiền"""
    print_separator("CHUYỂN TIỀN")
    print(f"Số dư hiện tại: {account.balance:,} VND")

    # Hiển thị danh sách tài khoản có thể chuyển
    print("\nDanh sách tài khoản:")
    for acc_num, acc in bank.accounts.items():
        if acc_num != account.account_number:
            print(f"- TK {acc_num}: {acc.account_holder}")

    target_account_num = get_number_input("\nNhập số tài khoản đích: ", 1)
    target_account = bank.find_account(target_account_num)

    if not target_account:
        print("❌ Không tìm thấy tài khoản đích!")
        wait_for_enter()
    return

```

```

if target_account.account_number == account.account_number:
    print("❌ Không thể chuyển tiền cho chính mình!")
    wait_for_enter()
    return

amount = get_number_input("Nhập số tiền muốn chuyển (VND): ", 1)
account.transfer(target_account, amount)
wait_for_enter()

def view_transaction_history(account):
    """Xem lịch sử giao dịch"""
    print_separator("LỊCH SỬ GIAO DỊCH")

    limit = get_number_input("Số giao dịch muốn xem (mặc định 10): ", 1)
    if limit == 0:
        limit = 10

    account.show_transaction_history(limit)
    wait_for_enter()

# Test tất cả functions
print("\n=== CHẠY CÁC TEST ===")
# Uncomment để test
# test_basic_features()
# test_bank_manager()
print("✅ Tất cả functions đã được hoàn thiện!")

```

## Phase 7: Hướng dẫn sử dụng và Demo

### Cách chạy chương trình:

1. Chạy tất cả các cell ở trên
2. Chạy cell cuối cùng để khởi động chương trình chính
3. Sử dụng menu để test các tính năng

### Tính năng đã hoàn thành:

- ✅ Class BankAccount với đầy đủ methods
- ✅ Class BankManager để quản lý nhiều tài khoản
- ✅ Hệ thống menu hoàn chỉnh
- ✅ Validation input từ người dùng
- ✅ Lịch sử giao dịch chi tiết
- ✅ Chuyển tiền giữa các tài khoản
- ✅ Thống kê ngân hàng

### Các tính năng nâng cao có thể thêm:

- 🚀 Lưu/đọc dữ liệu từ file

- 🚀 Tính lãi suất hàng tháng
- 🚀 Phân loại tài khoản (tiết kiệm, thanh toán)
- 🚀 Giới hạn giao dịch theo ngày
- 🚀 Mã PIN bảo mật
- 🚀 Export báo cáo PDF

## Bài tập mở rộng:

1. Thêm tính năng tạo mã PIN khi tạo tài khoản
2. Thêm giới hạn rút tiền tối đa mỗi ngày
3. Thêm tính năng tính lãi suất tự động
4. Thêm báo cáo thống kê theo thời gian

```
In [ ]: # Demo nhanh các tính năng
def quick_demo():
    print("\n🚀 === DEMO NHANH CÁC TÍNH NĂNG ===")

    # Tạo bank và tài khoản demo
    bank = BankManager()
    acc1 = bank.create_account("Alice", 1000000)
    acc2 = bank.create_account("Bob", 500000)

    print("\n1. Thông tin ban đầu:")
    bank.show_all_accounts()

    print("\n2. Alice nạp 200,000 VND:")
    acc1.deposit(200000)

    print("\n3. Alice chuyển 150,000 VND cho Bob:")
    acc1.transfer(acc2, 150000)

    print("\n4. Bob rút 100,000 VND:")
    acc2.withdraw(100000)

    print("\n5. Tình trạng cuối:")
    bank.show_all_accounts()

    print("\n6. Lịch sử giao dịch của Alice:")
    acc1.show_transaction_history()

    print("\n7. Thống kê ngân hàng:")
    print(f"💰 Tổng tiền: {bank.get_total_money():,} VND")
    print(f"👤 Tổng tài khoản: {len(bank.accounts)}")

# Chạy demo
quick_demo()

print("\n" + "="*60)
print("🎉 MINI PROJECT HOÀN THÀNH!")
print("Chạy main_program() để bắt đầu sử dụng hệ thống")
print("="*60)
```